FGV EMAp

Thalis Ambrosim Falqueto e João Pedro Jerônimo

Engenharia de Software

Revisão para A1

Rio de Janeiro

2026

Conteúdo

1 Documentação, Contratos e SRP 3
1.1 Cacto 4
1.2 Por que o `processar` deveria ser vários métodos (SRP) 8
1.3 Loja (exercício de correção) 8
1.4 Termos da Aula 1 10
2 Simple Factory, Factory Method e OCP 11
2.1 Simple Factory 12
2.2 Factory Method 13
2.3 Termos da Aula 2 15
3 Builder e Singleton 16
3.1 Builder 17
3.2 Singleton 20
3.2.1 Singleton no `__new__` 20
3.2.2 Singleton no decorador 21
3.3 Termos da Aula 3 21
4 Abstract Factory 23
4.1 Abstract Factory 24
4.1.1 Herança vs. composição 26
4.1.2 Por que se usa mais Abstract Factory do que Factory Method? 26
4.1.3 Voltando pra Loja 29
4.2 Termos da Aula 4 30

Documentação, Contratos e SRP

A aula usa dois exemplos: a “Cacto” é simplesmente uma loja de cactos com um método de quase 90 linhas que faz de tudo (aparentemente feito por algum aluno de LP), sem nenhuma documentação, e a “Loja” é o exercício de correção, que isola só a fatia de “calcular o frete” desse mesmo problema pra refatorar passo a passo, do jeito errado até o SRP resolvido.

1.1 Cacto

A loja vende cactos — cada um com nome, preço e altura — e monta pedidos com vários itens:

```python
class Cacto:
    def __init__(self, nome, preco, altura_cm):
        self.nome = nome
        self.preco = preco
        self.altura_cm = altura_cm


class ItemPedido:
    def __init__(self, cacto, quantidade):
        self.cacto = cacto
        self.quantidade = quantidade
```

Exemplo de **design by contract** citado em aula: o `__init__` de `ItemPedido` já declara, o que consome pra funcionar — um `cacto` e uma `quantidade`. Só que, como o próprio comentário no código admite, nada garante que `cacto` é de fato um `Cacto`. Documentar com docstring é pedir pra quem for usar a classe seguir o combinado: Python é fracamente tipado, então nada impede alguém de passar qualquer coisa ali. A saída ingênua seria verificar o tipo dentro do `__init__`, mas fazer esse tipo de verificação em todos os métodos e todos os parâmetros deixa o programa lento. Aí que entra o termo dito pelo Pinho: você declara o comportamento esperado e confia que quem chamou respeitou, em vez de reverificar tudo o tempo todo.

Na média, tudo que é interno ao sistema é tratado assim, e a verificação de fato só entra quando o sistema recebe algo de fora dele. Se um contrato não é respeitado, a consequência pode ser tão grave quanto o caso citado em aula do foguete da NASA que explodiu (não explodiu, mas **ver**).

Continuando o código, o pedido guarda os dados do cliente e a lista de itens:

```python
class Pedido:
    def __init__(self, cliente, email, cep, tipo_entrega, cupom, numero_cartao):
        self.cliente = cliente
        self.email = email
        self.cep = cep
        self.tipo_entrega = tipo_entrega
        self.cupom = cupom
        self.numero_cartao = numero_cartao
        self.itens = []
```

E a entrega tem quatro formas possíveis, todas implementando o mesmo contrato abstrato:

```python
class Entrega(ABC):
    @abstractmethod
    def calcular_frete(self, peso_kg, cep):
```

```
4           ...
5
6       @abstractmethod
7       def prazo_em_dias(self, cep):
8           ...
9
10      @abstractmethod
11      def codigo_rastreio(self):
12          ...
13
14
15  class Sedex(Entrega):
16      def calcular_frete(self, peso_kg, cep):
17          base = 18.90 + peso_kg * 2.35
18          if cep.startswith("6") or cep.startswith("7"):
19              base = base * 1.4
20          return base
21
22      def prazo_em_dias(self, cep):
23          return 2 if cep.startswith("0") else 5
24
25      def codigo_rastreio(self):
26          return "BR314159265BR"
27
28
29  class Drone(Entrega):
30      def calcular_frete(self, peso_kg, cep):
31          base = 49.90 + peso_kg * 8.0
32          if peso_kg > 3.0:
33              base = base + 60.0
34          return base
35
36      def prazo_em_dias(self, cep):
37          return 1
38
39      def codigo_rastreio(self):
40          return "DRN-8080"
41
42
43  class PomboCorreio(Entrega):
44      def calcular_frete(self, peso_kg, cep):
45          return 4.50 + peso_kg * 0.75
46
47      def prazo_em_dias(self, cep):
48          return
49
```

```
    def codigo_rastreio(self):
        return "OLHE-PARA-O-CEU"


class CarrocaDeBoi(Entrega):
    def calcular_frete(self, peso_kg, cep):  # nao usa o cep
        return 9.90 + peso_kg * 0.40

    def prazo_em_dias(self, cep):  # nao usa o cep
        return 30

    def codigo_rastreio(self):
        return "BOI-0404"
```

Tudo isso converge pro método onde o professor usa os termos como **code smell** e **god class**: `ProcessadorDePedido.processar` calcula subtotal, aplica desconto, calcula peso, decide a entrega, calcula frete e imposto, valida cartão, grava em "banco", manda e-mail, imprime nota fiscal e loga — tudo junto, num só lugar:

Python
```
class ProcessadorDePedido:
    def processar(self, pedido):
        subtotal = 0
        for item in pedido.itens:
            subtotal = subtotal + item.cacto.preco * item.quantidade

        desconto = 0
        if pedido.cupom is not None:
            if pedido.cupom == "NULLSAFE10":
                desconto = subtotal * 0.10
            elif pedido.cupom == "OFFBYONE5":
                desconto = 5.0
            elif pedido.cupom == "HELLOWORLD":
                desconto = subtotal * 0.15
                if desconto > 40.0:  # if devia ta pra fora desse elif
                    desconto = 40.0

        peso_kg = 0
        for item in pedido.itens:
            peso_kg = peso_kg + item.quantidade * (item.cacto.altura_cm * 0.05)

        if pedido.tipo_entrega == "SEDEX":
            entrega = Sedex()
        elif pedido.tipo_entrega == "DRONE":
            entrega = Drone()
        elif pedido.tipo_entrega == "POMBO":
            entrega = PomboCorreio()
        elif pedido.tipo_entrega == "CARROCA":
```

```
            entrega = CarrocaDeBoi()
        else:
            entrega = Sedex()  # talvez o sedex n entregue nesse lugar, tem q verificar

        frete = entrega.calcular_frete(peso_kg, pedido.cep)
        base_imposto = subtotal - desconto
        imposto = base_imposto * 0.18
        total = base_imposto + frete + imposto

        if pedido.numero_cartao is None or len(pedido.numero_cartao) != 16:
            raise ValueError("cartao invalido")
        soma = 0
        for c in pedido.numero_cartao:
            if c < "0" or c > "9":
                raise ValueError("cartao invalido")
            soma = soma + int(c)
        if soma % 10 != 0:
            raise ValueError("cartao recusado")

        sql = (
            "INSERT INTO pedidos (cliente, cep, entrega, total) VALUES ('"
            + pedido.cliente + "', '" + pedido.cep + "', '" + pedido.tipo_entrega
            + "', " + f"{total:.2f}" + ")"
        )
        print("[BANCO] " + sql)
        for item in pedido.itens:
            print(
                "[BANCO] INSERT INTO itens (pedido_cliente, cacto, qtd) VALUES ('"
                + pedido.cliente + "', '" + item.cacto.nome + "', "
                + str(item.quantidade) + ")"
            )

        corpo = "<html><body>"
        corpo = corpo + "<h1>Obrigado, " + pedido.cliente + "!</h1>"
        corpo = corpo + "<p>Seus cactos foram compilados sem warnings e entraram na fila de deploy.</p><ul>"
        for item in pedido.itens:
            corpo = corpo + "<li>" + str(item.quantidade) + "x " + item.cacto.nome + "</li>"
        corpo = corpo + "</ul><p>Frete: R$ " + f"{frete:.2f}" + "</p>"
        corpo = corpo + "<p>Total: R$ " + f"{total:.2f}" + "</p>"
        corpo = corpo + "<p>Previsao de entrega: " + str(entrega.prazo_em_dias(pedido.cep)) + " dias</p>"
        corpo = corpo + "</body></html>"
        print("[SMTP] enviando para " + pedido.email)
        print(corpo)
```

```
72
73          print("=== NOTA FISCAL ELETRONICA ===")
74          print("DESTINATARIO: " + pedido.cliente)
75          print("BASE DE CALCULO: " + f"{base_imposto:.2f}")
76          print("ICMS 18%: " + f"{imposto:.2f}")
77          print("VALOR TOTAL: " + f"{total:.2f}")
78          print("==============================")
79
80          print(
81              "[LOG] pedido de " + pedido.cliente + " processado com "
82              + str(len(pedido.itens)) + " itens, total " + f"{total:.2f}"
83          )
84
85          return total
```

Pinho também disse que ‘quanto mais você precisa dar scroll numa função, pior ela é’, ‘código longo com variáveis pouco significativas é ruim de manter’ e é ‘pouco provável que você realmente precise de uma classe com 1000 linhas’ (fazendo referência a um código em produção real).

1.2 Por que o `processar` deveria ser vários métodos (SRP)

O problema dessa função é que ela tem várias (funções). Se listassemos, quem, na empresa, poderia pedir uma mudança em `ProcessadorDePedido.processar` — e apontando exatamente onde, dentro do método, cada um bateria:

- a **contabilidade** precisar mudar a alíquota — o `0.18` fixo em `imposto = base_imposto * 0.18`;
- o **marketing** criar um cupom novo — o bloco `if pedido.cupom == "NULLSAFE10": ...`
- o **marketing** precisar mudar o e-mail — o bloco que monta `corpo` em HTML e manda pro “SMTP”;
- o **gateway do cartão** mudar — a validação de `numero_cartao` (tamanho, dígitos, soma);
- o **DBA** precisar mudar uma coluna — o `sql = "INSERT INTO pedidos ..."` montado por concatenação de string (que, à parte da aula, também é uma porta aberta pra SQL injection, já que `pedido.cliente` entra direto na query sem tratamento nenhum);
- o **SEFAZ** mudar o layout da nota — o bloco `"=== NOTA FISCAL ELETRONICA ==="`;
- os **Correios** mudarem o modo de calcular o peso — a fórmula `peso_kg = peso_kg + item.quantidade * (item.cacto.altura_cm * 0.05)`;
- o **COO** querer oferecer outra forma de envio — o `if/elif` de `tipo_entrega`.

É esse problema que o Princípio de Responsabilidade Única (SRP) corrige: cada componente deve apresentar uma única responsabilidade. É o primeiro dos cinco princípios do SOLID.

1.3 Loja (exercício de correção)

O professor exemplifica a solução numa loja simplificada que faz somente o frete, produto e recibo, pra refatorar passo a passo. A etapa 1 comete o mesmo erro numa escala menor:

```python
class Loja1:
    def processar(self, cliente, valor, peso_kg, centro):
        if centro == 'Sao Paulo':
            frete = 10.0 + 2.0 * peso_kg
        elif centro == 'Manaus':
            frete = 25.0 + 1.2 * peso_kg
```

```
        else:
            frete = 10.0 + 2.0 * peso_kg

        total = valor + frete
        return f'{cliente} pagou R$ {total:.2f} (envio de  {centro})'
```

A etapa 2 divide `Loja1` em peças, cada uma cuidando de uma única coisa — calcular o total, gerar o recibo, e decidir o custo do frete, com uma subclasse por tipo de frete:

```python
class Frete(ABC):
    @abstractmethod
    def custo(self, peso_kg):
        ...

class FreteRodoviario(Frete):
    def custo(self, peso_kg):
        return 10.0 + 2.0 * peso_kg

class FreteFluvial(Frete):
    def custo(self, peso_kg):
        return 25.0 + 1.2 * peso_kg

class CalculadoraTotal:
    def total(self, valor, frete):
        return valor + frete

class Recibo:
    def gerar(self, cliente, total, centro):
        return f'{cliente} pagou R$ {total:.2f} (envio de  {centro})'


class Loja2:
    def __init__(self, calculadora, recibo):
        self.calculadora = calculadora
        self.recibo = recibo

    def processar(self, cliente, valor, peso_kg, centro):
        if centro == 'Sao Paulo':
            frete = FreteRodoviario()
        elif centro == 'Manaus':
            frete = FreteFluvial()
        else:
            frete = FreteRodoviario()

        total = self.calculadora.total(valor, frete.custo(peso_kg))

        return self.recibo.gerar(cliente, total, centro)
```

Agora, `Loja2` ainda tem um `if`, decidindo qual objeto usar, mas não faz mais o cálculo do frete nem monta o recibo. Rodando as duas etapas lado a lado, o resultado é o mesmo, mas a segunda já está pronta pra crescer sem precisar reescrever tudo de novo:

```python
cliente = 'Ada'
valor = 100
peso_kg = 2

print('etapa 1 - tudo cagado')
loja1 = Loja1()
print(loja1.processar(cliente, valor, peso_kg, 'Sao Paulo'))
print(loja1.processar(cliente, valor, peso_kg, 'Manaus'))

print('etapa 2 - SRP dominado')
loja2 = Loja2(CalculadoraTotal(), Recibo())
print(loja2.processar(cliente, valor, peso_kg, 'Sao Paulo'))
print(loja2.processar(cliente, valor, peso_kg, 'Manaus'))
```

1.4 Termos da Aula 1

- **Docstring, typehint** - Vocês já sabem isso;
- **Design by contract** — declarar o comportamento esperado de um componente (o que ele espera receber, o que garante devolver) e confiar nisso, em vez de reverificar tudo em tempo de execução.
- **Defeito (fault/bug)** — falha concreta e localizável no código-fonte, que existe independente de ser executada.
- **Falha (failure)** — comportamento incorreto observável quando o programa roda e um defeito é de fato acionado.
- **Code smell** — sintoma de que o design do código está ruim, mesmo funcionando sem erros; o "cheiro" de um problema estrutural.
- **God class** — classe (ou método) que sabe e faz coisas demais, concentrando responsabilidades que deveriam estar separadas.
- **SRP (Princípio de Responsabilidade Única)** — cada componente deve ter um único motivo pra mudar; o S do SOLID.

Simple Factory, Factory Method e OCP

A solução da `Loja2` (Section 1.3) resolveu o SRP, mas o `if centro == ...` continua dentro de `processar`, decidindo qual `Frete` instanciar ainda hardcoded, ainda fazendo verificação de string, o que ainda é problema mesmo depois do SRP resolvido (Professor disse que deixar essas verificações em string é um crime, mas não vem ao caso). Some a isso uma regra de negócio nova: a loja quer simular o frete antes da compra, não só calculá-lo depois que o cliente já decidiu comprar, e passa a atender mais centros de distribuição. Dessa forma, agora, vários lugares diferentes vão precisar da mesma lógica de "qual frete usar para este centro", e ela está presa dentro de `Loja2`.

2.1 Simple Factory

Simple Factory resolve o problema das classes: tira o `if` de dentro da loja e concentra numa única classe. A ideia, resumida em aula, é simplesmente mudar o if de lugar:

```python
class FabricaDeFrete:
    def criar(self, centro):
        if centro == 'Sao Paulo':
            return FreteRodoviario()
        elif centro == 'Manaus':
            return FreteFluvial()
        else:
            return FreteRodoviario()


class SimuladorDeFrete:
    def __init__(self, fabrica):
        self.fabrica = fabrica

    def simular(self, centro, peso_kg):
        return self.fabrica.criar(centro).custo(peso_kg)


class Loja3:
    def __init__(self, fabrica, calculadora, recibo):
        self.fabrica = fabrica
        self.calculadora = calculadora
        self.recibo = recibo

    def processar(self, cliente, valor, peso_kg, centro):
        frete = self.fabrica.criar(centro)
        total = self.calculadora.total(valor, frete.custo(peso_kg))
        return self.recibo.gerar(cliente, total, centro)
```

Agora, a loja não precisa mais entender de frete. O ganho aparece em isolar o if do resto do código: o acoplamento entre `Loja` e a lógica de frete diminui. Pinho disse isso numa frase: 'toda vez que uma mudança no código não obriga o `main` a mudar junto, o design melhorou'. De fato, a etapa 3 do `main` só precisa montar a fábrica e injetá-la:

```python
print("ETAPA 3 - SRP simple factory")
fabrica = FabricaDeFrete()
```

```
loja3 = Loja3(fabrica, CalculadoraTotal(), Recibo())
print(loja3.processar(cliente, valor, peso_kg, 'Sao Paulo'))
print(loja3.processar(cliente, valor, peso_kg, 'Manaus'))
custo = SimuladorDeFrete(fabrica).simular("Belém", peso_kg)

print("Simulação Belém", custo)
```

Note que essa última linha ainda esconde o mesmo bug do `else`. Como `FabricaDeFrete.criar` só reconhece `'Sao Paulo'` e `'Manaus'`, simular pra “Belém” cai no `else` e devolve `FreteRodoviario`, o que pode não ser o certo pro caso.

Nesse ponto a aula sai do código e coloca o Simple Factory dentro de um vocabulário maior de arquitetura. O professor ajuda a definir alguns termos, que vou definir melhor aqui, não exatamente do jeito dito: um **padrão de projeto** é uma saída clássica, já testada, para um problema recorrente de modelagem; a referência dada foi o livro de 1994 do **GoF** (Gang of Four).

O GoF organiza os 23 padrões do livro de 1994 em três famílias: **padrões criacionais** (como criar objetos — Factory Method, Abstract Factory, Builder, Prototype, Singleton), **padrões estruturais** (como compor classes e objetos — Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy) e **padrões comportamentais** (como objetos interagem e distribuem responsabilidade — Observer, Strategy, State, Template Method, entre outros). O Simple Factory é um idioma didático, usado como degrau pra chegar no Factory Method — esse sim um dos criacionais reconhecidos pelo GoF.

2.2 Factory Method

O Simple Factory ainda tem um `if` central, só que concentrado num único lugar. Em vez de uma fábrica perguntar “qual centro é esse?”, cada centro de distribuição sabe, por si mesmo, qual frete criar — quem entrega essa decisão ao código passa a ser a própria classe do centro de distribuição. Isso é feito com um método abstrato que cada subclasse implementa à sua maneira, uma por centro (faz isso importando o decorador e passando o decorador a função e a classe a própria factory - é um método declarado sem corpo ou código de execução, que serve como uma regra obrigatória para que as classes filhas criem sua própria implementação). Vejamos no exemplo:

Python
```
class CentroDeDistribuicao(ABC):
    def __init__(self, cidade, calculadora, recibo):
        self.cidade = cidade
        self.calculadora = calculadora
        self.recibo = recibo

    @abstractmethod
    def criar_frete(self):
        ...

    def despachar(self, cliente, valor, peso_kg):
        frete = self.criar_frete()
        total = self.calculadora.total(valor, frete.custo(peso_kg))
        return self.recibo.gerar(cliente, total, self.cidade)

```

```python

class CentroSaoPaulo(CentroDeDistribuicao):
    def __init__(self, calculadora, recibo):
        super().__init__("São Paulo", calculadora, recibo)

    def criar_frete(self):
        return FreteRodoviario()


class CentroManaus(CentroDeDistribuicao):
    def __init__(self, calculadora, recibo):
        super().__init__("Manaus", calculadora, recibo)

    def criar_frete(self):
        return FreteFluvial()


class CentroBelem(CentroDeDistribuicao):
    def __init__(self, calculadora, recibo):
        super().__init__("Belém", calculadora, recibo)

    def criar_frete(self):
        return FreteFluvial()
```

Agora, Belém tem, por conta própria, o frete fluvial certo — sem precisar tocar em nenhum `if`. O restante do fluxo (calcular total, gerar recibo) fica implementado uma única vez, no método concreto `despachar` da classe-base — as subclasses só precisam dizer qual frete usar. O polimorfismo (significa “muitas formas” (no caso, a função criar_frete é a polimórfica)) substitui o `if`: quando o código chama `self.criar_frete()`, quem responde já é o objeto do centro certo, decidido no momento em que a classe foi escolhida (a instanciação), não dentro de um `if` em tempo de execução. Essa é a etapa 4:

```python
print("ETAPA 4")
sao_paulo = CentroSaoPaulo(CalculadoraTotal(), Recibo())
manaus = CentroManaus(CalculadoraTotal(), Recibo())
belem = CentroBelem(CalculadoraTotal(), Recibo())
print(sao_paulo.despachar(cliente, valor, peso_kg))
print(manaus.despachar(cliente, valor, peso_kg))
print(belem.despachar(cliente, valor, peso_kg))
```

O mesmo raciocínio vale pra produtos: pra cada produto novo que precise de fábrica própria, o padrão pede duas classes, o produto e a fábrica do produto (aqui, o “produto” é o `Frete`, e a “fábrica” é o próprio `CentroDeDistribuicao`).

Comparado ao Simple Factory, adicionar um centro de distribuição novo passa a significar **criar** uma subclasse, e não editar nenhuma existente: nada do código antigo precisou mudar pra isso, nem o `CentroDeDistribuicao` já existente precisou ser tocado pra nascer um tipo novo. Esse é,

literalmente, o enunciado do Open/Closed Principle, o O do SOLID: uma classe deve estar fechada para modificação e aberta para extensão.

2.3 Termos da Aula 2

- **Simple Factory** — classe cujo único trabalho é decidir qual objeto concreto criar, tirando essa decisão de quem consome o objeto.
- **Padrão de projeto (design pattern)** — solução clássica e já testada pra um problema recorrente de modelagem orientada a objetos; catalogados no livro do GoF (1994).
- **GoF (Gang of Four)** — apelido dos quatro autores do livro **Design Patterns** (1994).
- **Padrão criacional** — categoria de padrão de projeto que resolve "qual é a forma correta de criar um objeto" (Simple Factory, Factory Method, Builder, Singleton).
- **Factory Method** — cada subclasse decide, via método abstrato sobrescrito, qual objeto concreto criar; substitui o if por polimorfismo.
- **Polimorfismo** — objetos de subclasses diferentes respondem à mesma chamada de método, cada um à sua maneira.
- **OCP (Open/Closed Principle)** — uma classe deve estar fechada para modificação e aberta para extensão; o O do SOLID.

Builder e Singleton

3.1 Builder

O padrão de criação Builder fala sobre esconder a complexidade de montar um (ou vários (do mesmo)) objeto. O exemplo dado foi fazer uma requisição HTTP (você precisa de url, header, token, body), ou uma consulta SQL via ORM, com schema, select, from, where: linhas e mais linhas só pra montar uma chamada, o que não é gerenciável e dificulta a leitura pra quem lê depois.

Para resolver isso, cria-se uma classe específica só pra construir objetos de um tipo (nomeada normalmente como `{Produto}Builder`), onde cada decisão de criação complexa fica dentro dessa classe, e o objetivo é fazer decisões encadeadas (retornando o self a cada método do Builder).

O exemplo de aula foi montar um computador:

```python
class Computador:
    def __init__(self):
        self.processador = None
        self.memoria_gb = None
        self.armazenamento_gb = None
        self.placa_video = None

    def __str__(self):
        return f'Processador: {self.processador}, Memória: {self.memoria_gb}, Armazenamento: {self.armazenamento_gb} e Placa de Vídeo {self.placa_video}'


class ComputadorBuilder:
    def __init__(self):
        self.computador = Computador()

    def com_processador(self, processador):
        self.computador.processador = processador
        return self

    def com_memoria_gb(self, memoria_gb):
        self.computador.memoria_gb = memoria_gb
        return self

    def com_armazenamento_gb(self, armazenamento_gb):
        self.computador.armazenamento_gb = armazenamento_gb
        return self

    def com_placa_video(self, placa_video):
        self.computador.placa_video = placa_video
        return self

    def construir(self):
        return self.computador


```

```
computador_gamer = (
    ComputadorBuilder()
    .com_processador('Intel I7')
    .com_memoria_gb('16 GB RAM')
    .com_armazenamento_gb('1024')
    .com_placa_video('Nvd 5070')
    .construir()
)
```

Cada propriedade de construção difícil vira um método que devolve `self`, enquanto o resto fica no `__init__` de `Computador`. Todos os métodos do builder fazem isso, com exceção do `.construir()`, que é quem devolve o produto final já pronto. Na vida real você raramente monta um computador atributo por atributo toda vez: normalmente existe uma classe com as construções comuns já prontas (configurações padrão), reaproveitando o mesmo builder por baixo:

Python

```
# na vida real criamos objetos padrão
# muitas vezes vamos ter uma classe com todas as construções comuns que precisamos (configurações prontas)
class InfoCentro:
    def montar_computador_basico(self):
        return (
            ComputadorBuilder()
            .com_processador('Intel I3')
            .com_memoria_gb('8 GB RAM')
            .com_armazenamento_gb('256')
            .com_placa_video('Integrada')
            .construir()
        )

    def montar_computador_gamer(self):
        return (
            ComputadorBuilder()
            .com_processador('Intel I7')
            .com_memoria_gb('16 GB RAM')
            .com_armazenamento_gb('1024')
            .com_placa_video('Nvd 5070')
            .construir()
        )
```

Para esse exemplo, fiquei com a dúvida: qual a utilidade do Builder se eu podia simplesmente instanciar `Computador` passando as especificações direto (ou, se precisasse, colocar a classe `Computador` diretamente dentro de uma classe principal (como a `InfoCentro`))? Nesse caso, a resposta é: nenhuma, ou quase. `Computador(processador='Intel I7', memoria_gb='16 GB RAM', armazenamento_gb='1024', placa_video='Nvd 5070')` faz exatamente o mesmo que as cinco linhas encadeadas do `ComputadorBuilder`, com menos código e sem precisar de uma classe extra. O motivo é que o Builder clássico nasceu pra resolver o problema do construtor telescópico, em que pular um parâmetro opcional te obrigava a passar `null` pra todos os anteriores na ordem certa.

Nós não conseguimos enxergar esse problema pois o Python já resolve isso de fábrica com `kwargs` e parâmetros nomeados (colocar `parametro = valor_padrao`).

O Builder só ganha da alternativa direta quando o que está sendo montado não é só um punhado de campos escalares: é o caso dos dois exemplos que abriram a aula, montar um request HTTP ou uma query SQL, onde você vai **acumulando** uma quantidade variável de coisas (headers, cláusulas `where`) em vez de preencher campos fixos, ou quando um passo depende do resultado de outro (validar se a fonte de energia aguenta a placa de vídeo escolhida, por exemplo), ou quando você quer impedir que um objeto pela metade escape pro resto do código.

Um exemplo concreto onde o Builder faz sentido, voltando pro domínio da Cacto: montar a consulta de pedidos com um número variável de filtros, onde cada `.com_X()` só entra na query se o chamador realmente pedir aquele filtro.

Python

```python
class ConsultaPedidosBuilder:
    def __init__(self):
        self.tabela = "pedidos"
        self.condicoes = []
        self.parametros = []

    def com_cliente(self, cliente):
        self.condicoes.append("cliente = %s")
        self.parametros.append(cliente)
        return self

    def com_tipo_entrega(self, tipo_entrega):
        self.condicoes.append("tipo_entrega = %s")
        self.parametros.append(tipo_entrega)
        return self

    def com_total_minimo(self, total_minimo):
        self.condicoes.append("total >= %s")
        self.parametros.append(total_minimo)
        return self

    def construir(self):
        sql = f"SELECT * FROM {self.tabela}"
        if self.condicoes:
            sql += " WHERE " + " AND ".join(self.condicoes)
        return sql, self.parametros


sql, parametros = (
    ConsultaPedidosBuilder()
    .com_tipo_entrega("DRONE")
    .com_total_minimo(100.0)
    .construir()
)
```

```
35  # sql = 'SELECT * FROM pedidos WHERE tipo_entrega = %s AND total >= %s'
36  # parametros = ['DRONE', 100.0]
```

Um kwarg fixo não dá conta disso de forma limpa — teria que ser algo como `consultar(cliente=None, tipo_entrega=None, total_minimo=None)` com um `if` pra cada parâmetro checando se é `None` antes de colar no SQL, o que é exatamente a bagunça que o Builder organiza aqui. E de quebra, essa versão já resolve o problema de SQL injection apontado lá na Cacto: em vez de concatenar `pedido.cliente` direto na string, os valores viram parâmetros separados (`%s` + lista `parametros`), passados pro driver do banco em vez de virarem texto da query.

3.2 Singleton

Singleton parte de um problema oposto: em vez de facilitar criar vários objetos, ele impede que exista mais de um. Só podemos ter uma instância na aplicação, impossível ter um segundo objeto. A analogia dada em aula: é como colocar um assento onde só cabe uma pessoa. Um nome comum pra esse tipo de classe é `Config`, embora o exemplo em si não implemente nenhum método de negócio, só o mecanismo.

3.2.1 Singleton no `__new__`

A primeira forma de resolver isso funciona em qualquer linguagem orientada a objetos (existe outro jeito, mas só funciona em Python) e mexe direto no `__new__`:

Python

```python
from typing import ClassVar, Self


class SingletonNew:
    _instance: ClassVar[SingletonNew | None] = None

    def __new__(cls) -> Self:
        if cls._instance is None:
            print('[SingletonNew] Criando Nova instância')
            cls._instance = super().__new__(cls)
        else:
            print('[SingletonNew] Retornando instância existente!')

        return cls._instance

    def __init__(self):
        self.data: str = 'Shared Resource'


print(SingletonNew() is SingletonNew())
```

Essa solução também pode ser misturada com Builder e outros padrões — o código acima é só o esqueleto, poderia ter métodos de negócio como qualquer classe normal. Só que, como dito pelo Pinho, a solução não é boa: toda vez que você instancia `SingletonNew()`, o `__init__` roda de novo mesmo quando `__new__` está devolvendo a instância antiga — o estado se torna repetido sempre, mesmo sem criar um objeto novo. E pior: com herança, o Singleton se duplica. Se uma classe filha herda de uma classe mãe que já implementa esse `__new__`, a classe filha acaba pegando o

singleton da classe mãe. Não dá pra usar essa estratégia no `__new__` de forma confiável quando existe hierarquia.

3.2.2 Singleton no decorador

A segunda forma tenta resolver isso com um decorador, guardando cada instância criada num dicionário — em vez de uma cadeira, um banco com vários lugares, um pra cada tipo de classe decorada:

```python
from collections.abc import Callable
from typing import Any


def singleton_naive[T](cls: type[T]) -> Callable[..., T]:
    instances: dict[type[T], T] = {}

    def get_instance(*args: Any, **kwargs: Any) -> T:
        if cls not in instances:
            instances[cls] = cls(*args, **kwargs)
        return instances[cls]

    return get_instance


@singleton_naive
class SingletonNaive:
    '''Singleton sem uma preocupação terrível'''

    def __init__(self, rotulo: str = 'Sem algo que seria útil aqui') -> None:
        self.rotulo = rotulo


print(type(SingletonNaive))
print(SingletonNaive().__name__, SingletonNaive().__doc__)
```

O problema aqui é de outra natureza: `get_instance` deveria receber tudo que o construtor da classe original receberia, mas como o decorador funciona pra qualquer classe, ele não tem como saber de antemão o que cada uma espera. E tem um efeito colateral mais sério — depois do decorador, `SingletonNaive` deixa de ser uma classe e passa a ser uma função (`get_instance`, guardada como closure, com `cls` capturado como freevar no frame da função original). Isso significa que um `isinstance` contra `SingletonNaive` já não funciona mais do jeito esperado, porque você estaria comparando contra uma função, não contra um tipo. O singleton em si funciona perfeitamente, mas a classe não funciona mais perfeitamente como classe.

3.3 Termos da Aula 3

- **Builder** — padrão criacional que isola a lógica de montar um objeto complexo numa classe própria, com métodos encadeados.
- **Singleton** — padrão criacional que garante que uma classe tenha, no máximo, uma única instância na aplicação.

- **Closure** — função que “lembra” variáveis do escopo onde foi definida, mesmo depois que esse escopo termina de executar.
- **Freevar (variável livre)** — variável usada dentro de uma função mas definida fora dela, capturada pela closure.

Abstract Factory

A aula parte da ideia um app com widget de calendário e de clima, que precisa se comportar diferente dependendo do sistema (iPadOS ou iOS).

4.1 Abstract Factory

A solução pro problema de criar algo generalizável de acordo com o sistema é criar uma fábrica que produz, de uma vez, todos os widgets daquele sistema. Cada produto (calendário, clima) continua sendo decidido por um Factory Method, igual ao que já vimos, só que agora vários desses métodos moram juntos, na mesma fábrica:

```python
from abc import ABC, abstractmethod

class UIFactory(ABC):
    @abstractmethod
    def create_calendar(self): ...

    @abstractmethod
    def create_weather(self): ...

class iPadOSFactory(UIFactory):
    def create_calendar(self):
        print("Factory da Plataforma iPadOS: Calendário")
        widget = iPadOSCalendarWidget()
        widget.create_calendar()
        return widget

    def create_weather(self):
        print("Factory da Plataforma iPadOS: Clima")
        widget = iPadOSWeatherWidget()
        widget.create_weather()
        return widget

class iOSFactory(UIFactory):
    def create_calendar(self):
        print("Factory da Plataforma iOS: Calendário")
        widget = iOSCalendarWidget()
        widget.create_calendar()
        return widget

    def create_weather(self):
        print("Factory da Plataforma iOS: Clima")
        widget = iOSWeatherWidget()
        widget.create_weather()
        return widget


class CalendarWidget(ABC):
    @abstractmethod
```

```
39      def create_calendar(self): ...
40
41  class iPadOSCalendarWidget(CalendarWidget):
42      def create_calendar(self):
43          print("Criando um calendário para iPadOS")
44
45  class iOSCalendarWidget(CalendarWidget):
46      def create_calendar(self):
47          print("Criando um calendário para iOS")
48
49
50  class WeatherWidget(ABC):
51      @abstractmethod
52      def create_weather(self): ...
53
54  class iPadOSWeatherWidget(WeatherWidget):
55      def create_weather(self):
56          print("Criando um painel climático para iPadOS")
57
58  class iOSWeatherWidget(WeatherWidget):
59      def create_weather(self):
60          print("Criando um painel climático para iOS")
61
62
63  class ApplicationInterface:
64      def get_factory(self, platform_type):
65          if platform_type == "iPadOS":
66              return iPadOSFactory()
67          if platform_type == "iOS":
68              return iOSFactory()
69
70          raise ValueError("Esta plataforma não existe")
71
72
73  application_interface = ApplicationInterface()
74
75  print("***** iPadOS *****")
76  widget_factory = application_interface.get_factory("iPadOS")
77  widget_factory.create_calendar()
78  widget_factory.create_weather()
79
80  print("***** iOS *****")
81  widget_factory = application_interface.get_factory("iOS")
82  widget_factory.create_calendar()
83  widget_factory.create_weather()
```

(Reparo à parte: no material original da aula, `create_calendar` e `create_weather` da `UIFactory` criavam o widget e chamavam o método dele sem `return` — o widget criado era descartado, e `widget_factory.create_calendar()` sempre devolvia `None`. Funcionava pro demo porque tudo acontecia via `print`, mas quebrava o contrato usual de um Factory Method, que é devolver o produto pra quem pediu. Um `return` sozinho na frente da chamada não teria resolvido, porque o problema se repetia um nível abaixo — `iPadOSCalendarWidget().create_calendar()` também não tinha `return`. O código acima já está corrigido: cada método guarda o widget numa variável, chama o método dele pelo efeito colateral do `print`, e devolve o próprio widget — não o resultado de chamar `create_calendar()` nele. Assim `widget_factory.create_calendar()` devolve um `CalendarWidget` de verdade, que quem chamou pode guardar e usar depois. O exemplo de ciência de dados logo abaixo já seguia esse contrato desde o início: cada `create_X` devolve o objeto (`return PandasDatasetLoader()`), em vez de só imprimir e descartar.)

"Ao escolher uma fábrica, todo o restante do programa se adequa a ela": quem pede um widget a `iPadOSFactory` nunca recebe, por engano, um widget de iOS — os dois métodos vivem na mesma classe, então saem garantidamente da mesma família. `ApplicationInterface.get_factory` é o ponto único de decisão — mesma forma do `FabricaDeFrete.criar`, um `if` que devolve objetos diferentes.

Isso é **Abstract Factory**: "um padrão de projeto criacional que permite produzir famílias de objetos relacionados ou dependentes sem especificar suas classes concretas". E, como o próprio material registra, "as classes de Abstract Factory frequentemente são baseadas em um conjunto de Factory Methods" — `create_calendar` e `create_weather`, cada um isolado, já é um Factory Method; o Abstract Factory só os empacota juntos.

4.1.1 Herança vs. composição

A diferença entre os dois padrões não é a quantidade de métodos — é **como** a variação acontece. Factory Method varia por herança: pra trocar de família, você troca de classe. `CentroSaoPaulo` e `CentroManaus` são subclasses diferentes de `CentroDeDistribuicao`; a identidade da classe já carrega a decisão.

Abstract Factory varia por composição: a classe que usa a fábrica nunca muda. `ApplicationInterface` é uma classe só, do início ao fim — o que muda é qual objeto está guardado dentro dela (`iPadOSFactory()` ou `iOSFactory()`), recebido como retorno de um método, nunca por herança. É o mesmo mecanismo que já existia desde o Simple Factory (`Loja3` guardando `self.fabrica`, sem herdar de `FabricaDeFrete`) — só que agora a fábrica guardada tem vários métodos de criação em vez de um.

4.1.2 Por que se usa mais Abstract Factory do que Factory Method?

Fica a pergunta em aberto na aula: "se usa muito mais o abstract factory do que o factory method (porque?)". A resposta é que sistemas reais raramente precisam criar uma coisa isolada — quase sempre precisam de várias coisas que têm que combinar entre si. O segundo exemplo de aula, um pipeline de ciência de dados, mostra isso com três produtos em vez de dois — "mais uma classe com 3 métodos, a fábrica cria as mesmas classes, com as funções definidas antes":

```python
from abc import ABC, abstractmethod

# ABSTRAÇÕES (produtos) - o cliente conhece apenas isso

class DatasetLoader(ABC):
    @abstractmethod
```

```
7       def load(self) -> None: ...
8
9   class Model(ABC):
10      @abstractmethod
11      def train(self) -> None: ...
12
13  class Visualizer(ABC):
14      @abstractmethod
15      def plot(self) -> None: ...
16
17  # ABSTRAÇÃO DA FÁBRICA - contrato da família de produtos
18
19  class DataScienceFactory(ABC):
20      @abstractmethod
21      def create_dataset_loader(self) -> DatasetLoader: ...
22
23      @abstractmethod
24      def create_model(self) -> Model: ...
25
26      @abstractmethod
27      def create_visualizer(self) -> Visualizer: ...
28
29  # PRODUTOS CONCRETOS - Família Local (Pandas / Sklearn / Matplotlib)
30
31  class PandasDatasetLoader(DatasetLoader):
32      def load(self) -> None:
33          print("[PandasDatasetLoader] Lendo CSV local com pandas.read_csv(...).
            Amostras: 10_000")
34
35  class SklearnModel(Model):
36      def train(self) -> None:
37          print("[SklearnModel] Treinando LogisticRegression(solver=\"lbfgs\").
            Acurácia de validação: 0.89")
38
39  class MatplotlibVisualizer(Visualizer):
40      def plot(self) -> None:
41          print("[MatplotlibVisualizer] Plotando curva ROC e matriz de confusão com
            Matplotlib.")
42
43  # PRODUTOS CONCRETOS - Família Distribuída (Spark / MLlib / Seaborn)
44
45  class SparkDatasetLoader(DatasetLoader):
46      def load(self) -> None:
47          print("[SparkDatasetLoader] Lendo dados no cluster:
            spark.read.parquet(\"s3://bucket/dataset\"). Linhas: 120_000_000")
48
49  class MLlibModel(Model):
```

```
50      def train(self) -> None:
51          print("[MLlibModel] Treinando RandomForestClassifier em cluster (MLlib). AUC de validação: 0.92")
52
53  class SeabornVisualizer(Visualizer):
54      def plot(self) -> None:
55          print("[SeabornVisualizer] Gerando pairplot e heatmap de correlação com Seaborn.")
56
57  # FÁBRICAS CONCRETAS - produzem uma FAMÍLIA coerente de produtos
58
59  class LocalPandasFactory(DataScienceFactory):
60      """Família "local" para dados pequenos/medianos."""
61      def create_dataset_loader(self) -> DatasetLoader:
62          return PandasDatasetLoader()
63
64      def create_model(self) -> Model:
65          return SklearnModel()
66
67      def create_visualizer(self) -> Visualizer:
68          return MatplotlibVisualizer()
69
70  class DistributedSparkFactory(DataScienceFactory):
71      """Família "distribuída" para grandes volumes de dados."""
72      def create_dataset_loader(self) -> DatasetLoader:
73          return SparkDatasetLoader()
74
75      def create_model(self) -> Model:
76          return MLlibModel()
77
78      def create_visualizer(self) -> Visualizer:
79          return SeabornVisualizer()
80
81  # "SELETOR" DE FÁBRICA - ponto único de decisão concreta
82
83  class ApplicationInterface:
84      def get_factory(self, stack: str) -> DataScienceFactory:
85          if stack == "local":
86              return LocalPandasFactory()
87          if stack == "distributed":
88              return DistributedSparkFactory()
89          raise ValueError("Stack inválida. Use \"local\" ou \"distributed\".")
90
91
92  app = ApplicationInterface()
93
```

```
print("\n===== Cenário A: Pipeline LOCAL (dados pequenos/medianos) =====")
factory = app.get_factory("local")        # cliente recebe a FÁBRICA (abstração)
loader = factory.create_dataset_loader()  # abstração DatasetLoader
modelo = factory.create_model()           # abstração Model
viz = factory.create_visualizer()         # abstração Visualizer
loader.load()
modelo.train()
viz.plot()

print("\n===== Cenário B: Pipeline DISTRIBUÍDO (big data) =====")
factory = app.get_factory("distributed")  # troca a família, sem mudar o cliente
loader = factory.create_dataset_loader()
modelo = factory.create_model()
viz = factory.create_visualizer()
loader.load()
modelo.train()
viz.plot()
```

O cliente “só precisa conhecer as abstrações das nossas fábricas” — nunca importa `SklearnModel` nem `SparkDatasetLoader` diretamente, só `DataScienceFactory`. Fica também a régua de quando é seguro mexer: “o que pode mudar à vontade é acrescentar classes; classe concreta você não pode mudar” sem quebrar quem já depende dela — trocar o modelo de dentro de `LocalPandasFactory` é seguro; mudar a assinatura de `DataScienceFactory` (a abstração) afeta todo mundo que já depende dela.

4.1.3 Voltando pra Loja

A aula fecha puxando de volta pro exemplo que já vínhamos construindo: “se eu quiser ter uma calculadora de frete e recibo diferente pra cada estado, o que fazer?” A resposta sugerida nas anotações é dar um Factory Method pra cada peça que precisa variar (`criar_frete`, `criar_calculadora`, `criar_recibo`) e empacotar as três dentro de um Abstract Factory por centro. Um jeito de fazer isso, seguindo os dois exemplos acima:

Python

```
class ComponentesDeCentro(ABC):
    @abstractmethod
    def criar_frete(self): ...

    @abstractmethod
    def criar_calculadora(self): ...

    @abstractmethod
    def criar_recibo(self): ...


class ComponentesSaoPaulo(ComponentesDeCentro):
    def criar_frete(self):
        return FreteRodoviario()

```

```
    def criar_calculadora(self):
        return CalculadoraTotalSP()

    def criar_recibo(self):
        return ReciboSP()


class ComponentesManaus(ComponentesDeCentro):
    def criar_frete(self):
        return FreteFluvial()

    def criar_calculadora(self):
        return CalculadoraTotalAM()

    def criar_recibo(self):
        return ReciboAM()


class Loja4:
    def __init__(self, componentes):
        self.componentes = componentes

    def processar(self, cliente, valor, peso_kg, cidade):
        frete = self.componentes.criar_frete()
        calculadora = self.componentes.criar_calculadora()
        recibo = self.componentes.criar_recibo()
        total = calculadora.total(valor, frete.custo(peso_kg))
        return recibo.gerar(cliente, total, cidade)


loja = Loja4(ComponentesSaoPaulo())
print(loja.processar(cliente, valor, peso_kg, "Sao Paulo"))

loja = Loja4(ComponentesManaus())
print(loja.processar(cliente, valor, peso_kg, "Manaus"))
```

`CalculadoraTotalSP`/`CalculadoraTotalAM` e `ReciboSP`/`ReciboAM` são hipotéticos aqui — não existem nas classes já construídas, só ilustram a forma de como a alíquota de ICMS ou o layout de nota variariam por estado. `Loja4` nunca muda de classe entre um pedido de São Paulo e um de Manaus; o que muda é qual `Componentes` foi passado pro construtor.

E reaparece o OCP: “se a gente quiser colocar Belém no frete aéreo, pra fazer isso, teríamos que estender de `Frete`, sem precisar modificar, só estendendo — copia o `FreteFluvial` e bota `Aereo`” — a mesma `FreteAereo` que já ficou pronta, esperando, lá na Aula 2.

4.2 Termos da Aula 4

- **Abstract Factory** — padrão criacional que produz uma família inteira de objetos relacionados através de uma única fábrica, trocável por composição.

- **Composição** — relação “tem-um”: um objeto guarda uma referência a outro em vez de herdar dele; trocar o comportamento é trocar o objeto guardado, não a classe.
- **Família de produtos** — conjunto de objetos relacionados que precisam ser usados juntos e saem garantidamente compatíveis entre si, por virem da mesma fábrica concreta.